



UD1

ARQUITECTURA DE APLICACIONES WEB

MP_0613

Desarrollo web en entorno
servidor

1.1 Tipos de arquitectura

¿Qué es una aplicación web?

Una aplicación web es una aplicación a la que accedemos mediante una red, normalmente utilizando un navegador, y que permite al usuario realizar determinadas operaciones.

Por ejemplo, una tienda online permite:

- Consultar productos.
- Buscar información.
- Añadir productos a un carrito.
- Registrarse.
- Iniciar sesión.
- Realizar pedidos.

Aunque para el usuario parece que todo ocurre dentro del navegador, realmente intervienen diferentes componentes

¿Qué ocurre cuando accedemos a una página web?

Supongamos que escribimos:

<https://www.ejemplo.com/productos>

Y pulsamos **Enter** y de forma simplificada ocurre lo siguiente:



El navegador solicita información a un servidor y este devuelve una respuesta.

Por tanto, desde el principio debemos distinguir dos elementos fundamentales:

- **Cliente:** realiza la petición.
- **Servidor:** Recibe la petición y proporciona la respuesta.

Tipos de arquitectura

Desde la aparición de los primeros programas informáticos, tanto el desarrollo software como los paradigmas y modelos de programación han evolucionado enormemente. Las arquitecturas de desarrollo, o maneras en las que se organizan los diferentes actores en una aplicación, también han tenido una gran evolución a lo largo de estos años.

Dependiendo de la manera en la que se distribuyan las diversas capas de código de una aplicación, podemos **configurar diferentes arquitecturas**. De estas posibles combinaciones, podemos hacer una división en dos grandes grupos:

- Aplicaciones de escritorio o arquitectura local.
- Aplicaciones cliente servidor.

Arquitectura local o aplicaciones de escritorio

La arquitectura local es la arquitectura que siguen las aplicaciones tradicionales. Con tradicionales nos referimos a aplicaciones que **se ejecutan en el mismo equipo en el que se usan**. Los recursos a los que accede esta aplicación, como cheros, bases de datos, etc., se encuentran también dentro de la misma máquina, aunque podrían situarse en algún equipo externo y ser compartidos por otras aplicaciones.

Las aplicaciones de escritorio pueden ser adecuadas en entornos donde el programa vaya a ser manejado desde un **único equipo**. Pero, si necesitamos que varios equipos utilicen la aplicación, nos encontramos con algunos **inconvenientes**, entre los que destacamos:

- **Instalación:** la aplicación debe ser instalada en cada equipo en el que se vaya a utilizar, lo que en un entorno de muchos usuarios puede llegar a resultar una tarea pesada.
- **Requerimiento:** todos los equipos en los que se va a utilizar la aplicación deberán cumplir con los requerimientos software y hardware que esta necesite. Por ejemplo, si la aplicación solo funciona con cierto sistema operativo y necesita un mínimo de memoria RAM para funcionar, todos los equipos deberán disponer de estas características.

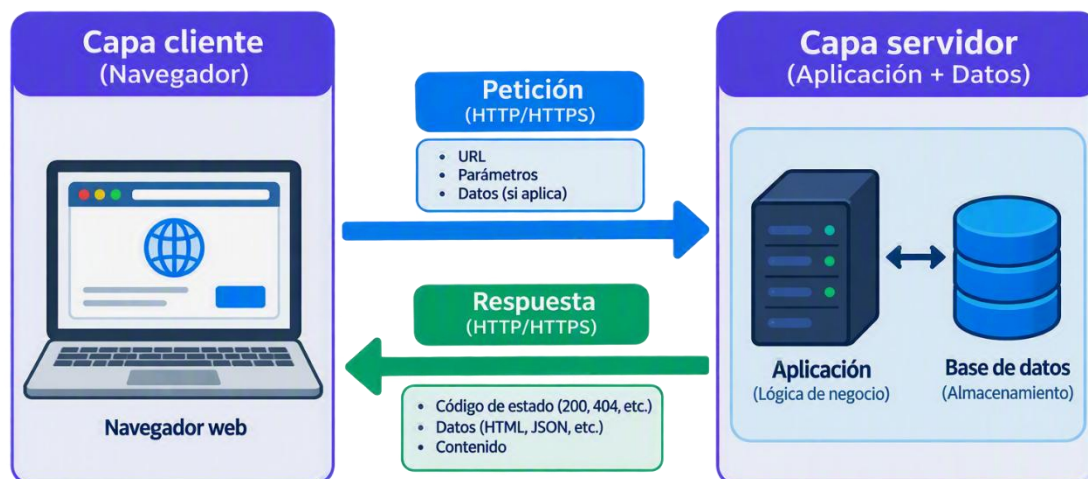
- **Reinstalación:** cualquier cambio en la aplicación requerirá reinstalarla en todos los equipos donde se vaya a utilizar.

Ejemplos de estos tipos de aplicaciones son los programas que normalmente utilizamos para ofimática, como **Microsoft Word, Excel** u otros que descargemos e instalemos en nuestro ordenador para hacer un uso local del mismo.

Arquitectura cliente - servidor

A diferencia de la arquitectura local, donde todo se centralizaba en el mismo equipo, la **arquitectura cliente-servidor** es un modelo en el que las funciones de una aplicación se distribuyen entre dos tipos principales de componentes: clientes y servidores.

El cliente solicita servicios y el servidor los proporciona.



- **Cliente:** El **cliente** es el componente que inicia la comunicación. En una aplicación web, normalmente el cliente es un navegador:
 - Google Chrome
 - Mozilla Firefox
 - Microsoft Edge
 - Safari

El navegador puede:

- Solicitar páginas web
- Enviar información al servidor.
- Recibir respuestas.
- Interpretar HTML.

- Aplicar CSS.
- Ejecutar JavaScript.
- Mostrar el resultado al usuario.

Por ejemplo, cuando rellenamos un formulario de registro, el navegador puede enviar al servidor los datos introducidos

RECUERDA:

El cliente solicita y el servidor responde

- **Servidor:** El **servidor** recibe las peticiones de los clientes y proporciona los recursos o servicios solicitados. En lugar de tener instalada la aplicación en cada equipo donde se va a utilizar, esta se **ejecuta en un ordenador remoto, al que conocemos como servidor**. Si la aplicación es utilizada por varios usuarios, cada usuario ejecutaría su propia instancia o hilo de la aplicación, pero todo estará centralizado en el mismo equipo

Puede encargarse de:

- Entregar páginas web.
- Ejecutar código.
- Consultar bases de datos.
- Validar información.
- Gestionar usuarios.
- Comprobar contraseñas.
- Procesar pedidos.
- Generar contenido dinámico.

Por ejemplo, cuando iniciamos sesión en una aplicación, el servidor puede recibir nuestro usuario y contraseña, comprobarlos y determinar si podemos acceder.

Cuando los datos se encuentran en otro equipo remoto estaríamos hablando de una variante del modelo cliente-servidor, conocida como **arquitectura de tres capas**.



Como vemos, tanto en el modelo cliente-servidor como en el de tres capas, **los clientes simplemente muestran la interfaz de usuario y recogen los datos, que son enviados al servidor para su procesamiento**. El servidor realiza las tareas de lógica de aplicación (cálculos, accesos a bases de datos, acceso a cheros, etc.), en función de los requerimientos del cliente y, posteriormente, le devuelve los resultados para que este los muestre.

Comunicación entre cliente y servidor: HTTP / HTTPS

Para que el cliente y el servidor puedan comunicarse necesitan utilizar un protocolo.

En las aplicaciones web, uno de los protocolos fundamentales es **HTTP (HyperText Transfer Protocol)**, utilizado para la comunicación entre clientes y servidores. Su versión segura, **HTTPS (HyperText Transfer Protocol Secure)**, añade una capa de cifrado que protege la información transmitida.

Estos protocolos establecen cómo se realizan las comunicaciones entre clientes y servidores web.

- **La petición:** El navegador realiza una **petición HTTP**.

Por ejemplo:

```
http
GET /index.html HTTP/1.1
Host: www.ejemplo.com
```

No es necesario memorizar todavía todos los elementos de una petición HTTP.

Lo importante es entender que el cliente está diciendo, en esencia:

“Quiero este recurso”

- **La respuesta:** El servidor procesa la petición y devuelve una **respuesta HTTP**.

Por ejemplo:

```
http
HTTP/1.1 200 ok
Content-Type: text/html
```

Después de las cabeceras puede aparecer el contenido solicitado.

Un código de respuesta habitual es:

Código	Significado
200	La petición se ha realizado correctamente
200	La petición se ha realizado correctamente
404	El recurso no se ha encontrado
403	Acceso prohibido
500	Error interno del servidor

Por ejemplo, cuando intentamos acceder a una página que no existe, normalmente recibiremos un error **404**.

Ventajas del modelo cliente-servidor

Entre las **ventajas** de este modelo frente a la arquitectura local podemos destacar:

- La aplicación **se instala en un único equipo**, que es el servidor o capa intermedia, facilitando su mantenimiento.
- Puesto que no tienen que ejecutar ningún código de la aplicación, **los clientes no necesitan disponer de ningún tipo de hardware o software especial**, solo la capacidad de comunicarse

con el servidor para enviarle datos y de presentar los resultados que la aplicación le devuelva.

- Si hay que realizar algún tipo de actualización en la aplicación, **solamente afectará al servidor**.

Resumen

- Existen diferentes tipos de arquitectura a la hora de construir aplicaciones informáticas.
- La arquitectura local es la arquitectura que siguen las aplicaciones tradicionales, es decir, aquellas que se instalan y se ejecutan en el mismo equipo en el que se utilizan.
- En la arquitectura cliente-servidor, por el contrario, hay dos elementos: el cliente y el servidor, y la aplicación puede estar alojada tanto en el equipo desde el que se ejecuta como en el servidor.
- La aplicación, además, puede estar alojada en un equipo remoto que no es el del cliente ni el del servidor. En este caso hablamos de una modalidad de la arquitectura cliente-servidor llamada 'arquitectura de tres capas'.
- La arquitectura cliente-servidor proporciona una serie de ventajas en aquellos casos en que queramos que la aplicación sea utilizada por múltiples ordenadores, ya que no exige ningún requisito hardware ni software especial a estos equipos, además de facilitar el mantenimiento y distribución de las aplicaciones.

Desventajas del modelo cliente-servidor

También hay puntos débiles que conviene conocer:

- **Dependencia de la conexión:** Si se pierde la red, el cliente no puede conectarse al servidor.
- **Punto único de fallo:** Si el servidor falla, todos los clientes se ven afectados.
- **Posible saturación:** Si demasiados usuarios hacen peticiones al mismo tiempo, el sistema puede ir más lento o bloquearse.
- **Actualizaciones de cliente:** Aunque el servidor centralice mucho, las aplicaciones del cliente pueden necesitar actualizaciones frecuentes.
- **Dificultad para detectar fallos:** No siempre es fácil saber si un problema viene del cliente o del servidor.
- **Los clientes no interactúan entre sí:** Toda la comunicación pasa por el servidor, lo que puede limitar ciertas funciones.

- **Riesgo de ciberataques:** Al estar siempre conectado y ser el punto central, el servidor puede ser objetivo de ataques si no está bien protegido.